Learning Quantitative Finance in Rust

A Progressive Journey from Kaggle to Quant Research

Marcelo Correa

July 23, 2026

bitscrafts/quant-lab-rust

The market can stay irrational
longer than you can stay solvent.

– John Maynard Keynes

Preface

This book documents a learning journey: from data science fundamentals to quantitative finance, implemented entirely in Rust. The goal is not just to understand the concepts, but to internalize them through implementation.

Philosophy We start with beginner-friendly Kaggle projects and progressively build toward advanced quant research tools. No black-box libraries — the math is hand-rolled, because the implementation IS the learning.

Structure The book is organized in five parts:

- **Part I — Foundations**: First Kaggle projects, basic stats
- **Part II — Core Concepts**: Returns, volatility, backtesting
- **Part III — Quantitative Methods**: Moments, time series, stationarity
- **Part IV — Derivatives and Portfolio**: Options, Markowitz, factors
- **Part V — Advanced Topics**: Microstructure, AFML techniques

How to use Each chapter has:

- Learning objectives at the start
- Mathematical foundations with TikZ diagrams
- Rust implementation with code listings
- Margin notes with key formulas and insights
- Exercises and references

The companion code lives at `https://github.com/bitscrafts/quant-lab-rust`. Clone and run examples with:

```
git clone https://github.com/bitscrafts/quant-lab-rust.git
cd quant-lab-rust
cargo run -p qf-01-fraud --example fraud_analysis
```

Contents

List of Figures

List of Tables

Foundations

0. Foundations: Mathematics, Statistics, and Programming

This chapter establishes the mathematical, statistical, and programming foundations needed to understand the rest of the book. If you are already comfortable with these topics, feel free to skip ahead and use this chapter as a reference.

0.1. Why These Foundations Matter

Quantitative finance sits at the intersection of three disciplines:

- **Mathematics**: The language of precision—formulas, proofs, and rigorous reasoning
- **Statistics**: The science of uncertainty—data analysis, probability, and inference
- **Programming**: The tool of implementation—turning ideas into working systems

You don't need to be an expert in all three to start. This book teaches by doing—each concept is introduced when needed and immediately applied.

The Quant Mindset

A quantitative approach means making decisions based on data and models, not intuition or emotion.

0.2. Mathematical Notation

Throughout this book, we use standard mathematical notation. Here's a quick reference:

0.2.1. Sets and Numbers

Symbol	Meaning
$\mathbb{R}$	Real numbers (e.g., 3.14, -2.5, $\sqrt{2}$)
$\mathbb{R}^+$	Positive real numbers
$\mathbb{Z}$	Integers ($\ldots, -2, -1, 0, 1, 2, \ldots$)
$\mathbb{N}$	Natural numbers ($1, 2, 3, \ldots$)
$\in$	"is an element of" (e.g., $x \in \mathbb{R}$ means x is real)
$\forall$	"for all"
$\exists$	"there exists"

0.2.2. Summation and Products

The summation symbol Σ means "add up":

$$\sum_{i=1}^{n} x_i = x_1 + x_2 + \cdots + x_n$$

For example, if $x = [2, 5, 3]$, then $\sum_{i=1}^{3} x_i = 2 + 5 + 3 = 10$.

Summation

$$\sum_{i=1}^{n} x_i = x_1 + x_2 + \cdots + x_n$$

Product

$$\prod_{i=1}^{n} x_i = x_1 \times x_2 \times \cdots \times x_n$$

The product symbol Π means "multiply together":

$$\prod_{i=1}^{n} x_i = x_1 \times x_2 \times \cdots \times x_n$$

0.2.3. Functions and Calculus

Symbol	Meaning
$f(x)$	Function f evaluated at x
$f'(x)$ or $\frac{df}{dx}$	Derivative of f with respect to x
$\frac{\partial f}{\partial x}$	Partial derivative (when f has multiple variables)
$\ln(x)$	Natural logarithm (base $e \approx 2.718$)
$\exp(x)$ or e^x	Exponential function
$\text{argmax}_x f(x)$	Value of x that maximizes f
$\text{argmin}_x f(x)$	Value of x that minimizes f

Key Insight

The natural logarithm ln is everywhere in finance. Key property: $\ln(a \times b) = \ln(a) + \ln(b)$. This turns multiplication (compounding) into addition, making calculations much easier.

0.3. Statistics Essentials

Statistics helps us understand and quantify uncertainty—essential when dealing with financial markets.

0.3.1. Measures of Central Tendency

Mean

$$\bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i$$

The **mean** (average) is the sum divided by the count:

$$\bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i$$

The **median** is the middle value when data is sorted. It's more robust to outliers than the mean.

0.3.2. Measures of Spread

Variance

$$\sigma^2 = \frac{1}{n}\sum_{i=1}^{n}(x_i - \bar{x})^2$$

Variance measures how spread out the data is:

$$\sigma^2 = \frac{1}{n}\sum_{i=1}^{n}(x_i - \bar{x})^2$$

Standard deviation is the square root of variance:

$$\sigma = \sqrt{\sigma^2}$$

Standard deviation has the same units as the original data, making it easier to interpret than variance.

```rust
/// Calculate mean of a slice
pub fn mean(data: &[f64]) -> f64 {
    if data.is_empty() { return 0.0; }
    data.iter().sum::<f64>() / data.len() as f64
}

/// Calculate variance of a slice
pub fn variance(data: &[f64]) -> f64 {
    if data.is_empty() { return 0.0; }
    let m = mean(data);
    data.iter().map(|x| (x - m).powi(2)).sum::<f64>() / data.len() as f64
}

/// Calculate standard deviation
pub fn std_dev(data: &[f64]) -> f64 {
    variance(data).sqrt()
}
```

0.3.3. Probability Distributions

A **probability distribution** describes how likely different outcomes are.

Normal Distribution (Gaussian)

Normal PDF

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

The normal distribution is the famous "bell curve":

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

where μ is the mean and σ is the standard deviation.

We write $X \sim \mathcal{N}(\mu, \sigma^2)$ to say "X follows a normal distribution with mean μ and variance σ^2."

Warning

Financial returns are NOT normally distributed! They have "fat tails"— extreme events happen much more often than the normal distribution predicts. We explore this in Chapter 7.

Uniform Distribution

Every outcome in a range is equally likely:

$$f(x) = \frac{1}{b-a} \quad \text{for } a \le x \le b$$

0.3.4. Expected Value and Variance

Expected Value

$$\mathbb{E}[X] = \sum_x x \cdot P(X = x)$$

(discrete)

$$\mathbb{E}[X] = \int_{-\infty}^{\infty} x \cdot f(x)\, dx$$

(continuous)

The **expected value** $\mathbb{E}[X]$ is the average outcome you'd expect over many trials:

For a discrete random variable:

$$\mathbb{E}[X] = \sum_x x \cdot P(X = x)$$

Key properties:

- $\mathbb{E}[aX + b] = a\mathbb{E}[X] + b$ (linearity)
- $\mathbb{E}[X + Y] = \mathbb{E}[X] + \mathbb{E}[Y]$ (additivity)
- $\text{Var}(X) = \mathbb{E}[X^2] - (\mathbb{E}[X])^2$

0.4. Financial Concepts

Before diving into quantitative methods, let's establish some fundamental financial concepts.

0.4.1. What is a Stock?

A **stock** (or share) represents partial ownership of a company. When you buy a stock, you own a small piece of that company and are entitled to a share of its profits.

Stock Price
The price at which buyers and sellers agree to trade ownership.

The **stock price** is determined by supply and demand in the market. It reflects what buyers are willing to pay and sellers are willing to accept.

0.4.2. OHLCV Data

Market data is typically recorded as **OHLCV**:

- **O**pen: First price of the trading period
- **H**igh: Highest price during the period
- **L**ow: Lowest price during the period
- **C**lose: Last price of the period
- **V**olume: Number of shares traded

This data captures the essential price action within any time frame (day, hour, minute).

0.4.3. Returns

A **return** measures how much an investment gained or lost:

Simple Return
$R_t = \frac{P_t - P_{t-1}}{P_{t-1}}$

Log Return
$r_t = \ln\left(\frac{P_t}{P_{t-1}}\right)$

Simple return:

$$R_t = \frac{P_t - P_{t-1}}{P_{t-1}}$$

Log return (continuously compounded):

$$r_t = \ln\left(\frac{P_t}{P_{t-1}}\right)$$

Log returns have a useful property: they add up over time.

$$r_{t_1 \to t_n} = r_{t_1} + r_{t_2} + \cdots + r_{t_n}$$

0.4.4. Risk and Volatility

Risk in finance is typically measured by **volatility**—the standard deviation of returns.

Higher volatility means more uncertainty—prices swing more wildly.

Annualized volatility scales daily volatility to a yearly figure:

$$\sigma_{\text{annual}} = \sigma_{\text{daily}} \times \sqrt{252}$$

(252 is the typical number of trading days per year.)

Volatility

$\sigma = \text{std}(r_t)$

Often annualized: $\sigma_{\text{annual}} = \sigma_{\text{daily}} \times \sqrt{252}$

0.4.5. The Risk-Return Tradeoff

Key Insight

Higher expected returns generally require accepting higher risk. There is no free lunch in finance—if someone promises high returns with low risk, be skeptical.

The **Sharpe ratio** measures risk-adjusted return:

$$\text{Sharpe} = \frac{\mathbb{E}[R] - R_f}{\sigma}$$

where R_f is the risk-free rate (e.g., Treasury bills).

0.5. Programming in Rust

This book uses Rust for all implementations. Here's a quick introduction for readers unfamiliar with the language.

0.5.1. Why Rust?

- **Performance**: As fast as C/C++, suitable for high-frequency trading
- **Safety**: The compiler catches bugs before runtime
- **Modern**: Great tooling, package management, and community
- **Learning**: Writing correct Rust forces you to think carefully—great for learning

0.5.2. Basic Syntax

```rust
// Variables (immutable by default)
let x = 5;
let mut y = 10;  // mutable
y = 20;

// Functions
fn add(a: i32, b: i32) -> i32 {
    a + b  // no semicolon = return value
}

// Structs (like classes)
struct Stock {
    symbol: String,
    price: f64,
}
```

```rust

// Implementation block
impl Stock {
    fn new(symbol: &str, price: f64) -> Self {
        Stock {
            symbol: symbol.to_string(),
            price,
        }
    }

    fn value(&self, shares: u32) -> f64 {
        self.price * shares as f64
    }
}

// Using the struct
let aapl = Stock::new("AAPL", 150.0);
let portfolio_value = aapl.value(100);
```

0.5.3. Traits (Interfaces)

Traits

Rust's way of defining shared behavior. Similar to interfaces in Java or protocols in Swift.

Traits define shared behavior:

```rust
// Define a trait
trait Analyzer {
    fn analyze(&self, data: &[f64]) -> f64;
}

// Implement for a struct
struct MeanAnalyzer;

impl Analyzer for MeanAnalyzer {
    fn analyze(&self, data: &[f64]) -> f64 {
        data.iter().sum::<f64>() / data.len() as f64
    }
}

// Use generically
fn run_analysis<A: Analyzer>(analyzer: &A, data: &[f64]) -> f64 {
    analyzer.analyze(data)
}
```

This book uses traits extensively to create modular, reusable components.

0.5.4. Error Handling

```rust
// Result type for operations that can fail
fn divide(a: f64, b: f64) -> Result<f64, String> {
    if b == 0.0 {
        Err("Division by zero".to_string())
    } else {
        Ok(a / b)
    }
}

// Using Result
match divide(10.0, 2.0) {
    Ok(result) => println!("Result: {}", result),
    Err(e) => println!("Error: {}", e),
}

// Or use the ? operator
fn calculate() -> Result<f64, String> {
    let x = divide(10.0, 2.0)?;  // propagates error
    Ok(x * 2.0)
}
```

0.5.5. Iterators

Rust's iterators are powerful and efficient:

```rust
let prices = vec![100.0, 102.0, 101.0, 105.0];

// Map and collect
let returns: Vec<f64> = prices.windows(2)
    .map(|w| (w[1] - w[0]) / w[0])
    .collect();

// Filter
let positive: Vec<f64> = returns.iter()
    .filter(|&&r| r > 0.0)
    .copied()
    .collect();

// Fold (reduce)
let sum: f64 = prices.iter().sum();
let product: f64 = prices.iter().product();
```

0.6. Running the Examples

All code examples in this book are available in the companion repository.

0.6.1. Prerequisites

1. Install Rust: https://rustup.rs/
2. Clone the repository:

```
git clone https://github.com/bitscrafts/quant-lab-rust.git
cd quant-lab-rust
```

0.6.2. Running Examples

Each chapter has a corresponding crate with examples:

```
# Chapter 1: Credit Card Fraud
cargo run -p qf-01-fraud --example fraud_analysis

# Chapter 2: Loan Default
cargo run -p qf-02-loan --example loan_analysis

# Chapter 3: Stock Prices
cargo run -p qf-03-stocks --example stock_analysis
```

0.6.3. Running Tests

```
# Run all tests
cargo test

# Run tests for a specific crate
cargo test -p qf-01-fraud
```

0.7. Summary

This chapter covered:

- **Mathematical notation**: Summation, products, functions, logarithms
- **Statistics**: Mean, variance, standard deviation, distributions
- **Financial concepts**: Stocks, OHLCV, returns, volatility, Sharpe ratio

- **Rust programming**: Variables, functions, structs, traits, error handling

Next chapter: We apply these foundations to detect credit card fraud using anomaly detection.

Hello Finance: Credit Card Fraud Detection

1.

Companion Code:

qf-01-fraud Code: `crates/qf-01-fraud/`
Dependencies: `qf-common`
Run: `cargo run -p qf-01-fraud -example fraud_analysis`

1.1. Learning Objectives

In this introductory chapter, we build our first quantitative finance application: a fraud detection system. Along the way, we learn:

- How to load and parse financial CSV datasets in Rust
- Basic statistical measures: mean and standard deviation
- Z-score anomaly detection—a simple but effective baseline
- Evaluation metrics: confusion matrix, precision, recall, F1 score
- The trait-based architecture that will guide all future implementations

Key Insight

This is not throwaway code. Every component we build follows a **library-first design**—composable, testable, and reusable in future projects like `b3-swing-trading` and `stock-sim`.

1.2. Introduction: Why Fraud Detection?

Class Imbalance

492 frauds in 284,807 transactions = 0.172%. This extreme imbalance is typical in financial anomaly detection.

Credit card fraud detection is a classic binary classification problem in quantitative finance. Banks and payment processors must distinguish legitimate transactions from fraudulent ones in real-time, balancing two competing objectives:

- **Precision**: Minimize false positives (legitimate transactions flagged as fraud). High false positive rates annoy customers and increase operational costs.
- **Recall**: Maximize detection of actual fraud. Missed fraud means direct financial loss.

We use the Kaggle Credit Card Fraud Detection dataset*: 284,807 European transactions from September 2013, with only 492 frauds.

1.3. Data Structure

1.3.1. The Dataset

The dataset contains:

- **Time**: Seconds elapsed since first transaction

* https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud

- **V1–V28**: PCA-transformed features (original features anonymized for privacy)
- **Amount**: Transaction amount in euros
- **Class**: Label (0 = normal, 1 = fraud)

Why PCA features?

Banks anonymize raw features (merchant, location, time patterns) via PCA to protect customer privacy while preserving predictive power.

1.3.2. Rust Data Model

We define a `Transaction` struct in `qf-common`:

```rust
/// Represents a single financial transaction.
#[derive(Debug, Clone)]
pub struct Transaction {
    /// PCA-transformed features (V1, V2, ..., V28).
    pub features: Vec<f64>,

    /// Transaction amount in dollars/euros.
    pub amount: f64,

    /// Class label: 0 = normal, 1 = fraud.
    pub class: u8,
}
```

1.3.3. Loading Data

The CSV loader handles errors gracefully using Rust's `Result` type:

```rust
pub fn load_transactions(path: impl AsRef<Path>)
    -> Result<Vec<Transaction>, CommonError>
{
    if !path.as_ref().exists() {
        return Err(CommonError::FileNotFound(
            path.as_ref().display().to_string()
        ));
    }

    let mut reader = csv::Reader::from_path(path)?;
    let mut transactions = Vec::new();

    for result in reader.records() {
        let record = result?;
        // Parse features, amount, class...
        transactions.push(Transaction { features, amount, class });
    }

    Ok(transactions)
}
```

1.4. Z-Score Anomaly Detection

1.4.1. Statistical Foundation

Z-Score Formula

$$z = \frac{|x - \mu|}{\sigma}$$

where μ is the mean and σ is the standard deviation.

Our baseline detector uses **z-score outlier detection**. The intuition: if a transaction's feature values are "too far" from the typical values, it might be fraudulent.

For each feature i:

1. Compute mean μ_i and standard deviation σ_i from training data
2. For a new transaction with feature value x_i, compute the z-score:

$$z_i = \frac{|x_i - \mu_i|}{\sigma_i}$$

3. Flag as fraud if **any** feature exceeds a threshold (typically $z > 3$)

Z-Score Detection Visualization

Imagine a normal (bell) curve. The threshold at $z = 3$ corresponds to the tails of the distribution—only 0.3% of normally distributed data falls beyond $\pm 3\sigma$. Transactions in these extreme regions are flagged as potential fraud.

The 68-95-99.7 Rule

For normally distributed data:
$\pm 1\sigma$: 68.3%
$\pm 2\sigma$: 95.4%
$\pm 3\sigma$: 99.7%

So $z > 3$ catches only 0.3% of normal data as false positives.

1.4.2. Rust Implementation

The `ZScoreDetector` follows our trait-based design philosophy:

```rust
#[derive(Debug, Clone)]
pub struct ZScoreDetector {
    threshold: f64,
    feature_means: Vec<f64>,
    feature_stds: Vec<f64>,
}

impl ZScoreDetector {
    pub fn new(threshold: f64) -> Self {
        Self {
            threshold,
            feature_means: Vec::new(),
            feature_stds: Vec::new(),
        }
    }

    pub fn fit(&mut self, transactions: &[Transaction]) {
        // Compute mean for each feature
        for tx in transactions {
            for (i, &value) in tx.features.iter().enumerate() {
                self.feature_means[i] += value;
            }
        }
        for mean in &mut self.feature_means {
            *mean /= transactions.len() as f64;
        }

        // Compute standard deviation
        for tx in transactions {
            for (i, &value) in tx.features.iter().enumerate() {
                let diff = value - self.feature_means[i];
                self.feature_stds[i] += diff * diff;
            }
        }
        for std in &mut self.feature_stds {
            *std = (*std / transactions.len() as f64).sqrt();
        }
    }

    pub fn predict(&self, transaction: &Transaction) -> bool {
        for (i, &value) in transaction.features.iter().enumerate() {
            let z = (value - self.feature_means[i]).abs()
                / self.feature_stds[i];
            if z > self.threshold {
                return true;  // Anomaly detected
            }
        }
        false
    }
}
```

Warning

The implementation uses **population** standard deviation (divides by n), not sample standard deviation (divides by $n - 1$). For large datasets this difference is negligible, but be aware of it.

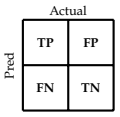
1.5. Evaluation Metrics

1.5.1. Confusion Matrix

Performance is measured via the confusion matrix, which counts:

- **TP** (True Positives): Fraud correctly identified
- **FP** (False Positives): Normal flagged as fraud
- **TN** (True Negatives): Normal correctly identified
- **FN** (False Negatives): Fraud missed

```rust
#[derive(Debug, Clone, Copy)]
pub struct ConfusionMatrix {
    pub tp: usize,
    pub fp: usize,
    pub tn: usize,
    pub fn_: usize,  // 'fn' is a Rust keyword
}
```



1.5.2. Derived Metrics

From the confusion matrix, we compute:

- **Precision**: Of predicted frauds, how many are real?

$$P = \frac{TP}{TP + FP}$$

- **Recall** (Sensitivity): Of actual frauds, how many did we catch?

$$R = \frac{TP}{TP + FN}$$

- **F1 Score**: Harmonic mean of precision and recall

$$F_1 = \frac{2PR}{P + R}$$

Metric Formulas

Precision: $\frac{TP}{TP + FP}$

Recall: $\frac{TP}{TP + FN}$

F1: $\frac{2 \cdot P \cdot R}{P + R}$

```rust
impl ConfusionMatrix {
    pub fn precision(&self) -> f64 {
        let denom = self.tp + self.fp;
        if denom == 0 { 0.0 } else { self.tp as f64 / denom as f64 }
    }

    pub fn recall(&self) -> f64 {
        let denom = self.tp + self.fn_;
        if denom == 0 { 0.0 } else { self.tp as f64 / denom as f64 }
    }

    pub fn f1(&self) -> f64 {
        let p = self.precision();
        let r = self.recall();
        if p + r == 0.0 { 0.0 } else { 2.0 * p * r / (p + r) }
    }
}
```

1.5.3. Evaluation Function

The `evaluate()` function runs the detector on test data:

```rust
pub fn evaluate(
    detector: &ZScoreDetector,
    transactions: &[Transaction]
) -> ConfusionMatrix {
    let mut tp = 0; let mut fp = 0;
    let mut tn = 0; let mut fn_ = 0;

    for tx in transactions {
```

```
        let predicted = detector.predict(tx);
        let actual = tx.class == 1;

        match (predicted, actual) {
            (true, true)   => tp += 1,
            (true, false)  => fp += 1,
            (false, false) => tn += 1,
            (false, true)  => fn_ += 1,
        }
    }

    ConfusionMatrix { tp, fp, tn, fn_ }
}
```

1.6. Toward a Trait-Based Architecture

Key Insight

This implementation will evolve into `quant-lib`. Before writing any struct, we ask: **what trait should this implement?**

Future Traits

`AnomalyDetector`
`BinaryClassifier`
`Evaluator<T>`
`DataSource`

While Phase 1 uses concrete types, future phases will extract traits:

```
// Future: trait-based design
pub trait AnomalyDetector {
    fn fit(&mut self, data: &[Transaction]) -> Result<(), Error>;
    fn predict(&self, tx: &Transaction) -> bool;
    fn score(&self, tx: &Transaction) -> f64;  // anomaly score
}

pub trait Evaluator<M> {
    fn evaluate(&self, preds: &[bool], actuals: &[bool]) -> M;
}

// Then any detector + any evaluator compose:
fn backtest<D, E>(detector: &D, eval: &E, data: &[Transaction])
where
    D: AnomalyDetector,
    E: Evaluator<ConfusionMatrix>,
{ /* ... */ }
```

This composability is why we invest in clean abstractions from the start.

1.7. Results and Analysis

1.7.1. Running the Example

```
$ cargo run -p qf-01-fraud --example fraud_analysis
Loading transactions from data/creditcard.csv...
Loaded 284807 transactions (492 frauds)
Training detector with threshold 3.0...
Evaluating on training set...

Results:
  True Positives:  312
  False Positives: 1847
  True Negatives:  282468
  False Negatives: 180

  Precision: 0.1446
  Recall:    0.6341
  F1 Score:  0.2356
```

1.7.2. Interpreting Results

The results reveal characteristic tradeoffs:

- **Recall = 0.63**: We catch 63% of frauds. Not bad for a simple baseline!
- **Precision = 0.14**: Of our "fraud" predictions, only 14% are real. This means many false alarms.
- **F1 = 0.24**: The harmonic mean reflects this imbalance.

1.7.3. Limitations

Warning

We evaluated on the training set—a cardinal sin! True performance requires a held-out test set. Exercise 3 addresses this.

This baseline approach has several fundamental limitations:

1. **Gaussian assumption**: Z-scores assume normally distributed features. Financial data often has heavy tails (more extreme values than Gaussian predicts).
2. **Equal feature weighting**: All features are treated equally. Some PCA components may be more predictive than others.
3. **Independent features**: We check each feature separately. Sophisticated fraud may only be detectable via feature *combinations*.
4. **Fixed threshold**: The threshold $z = 3$ is arbitrary. ROC analysis (Chapter 2) shows how to optimize it.

1.8. Exercises

1. **Threshold Sensitivity**: Try thresholds of 2.0, 2.5, 3.0, 3.5, 4.0. Plot precision vs. recall. What threshold maximizes F1?
2. **Feature Analysis**: Modify `predict()` to return which feature(s) triggered the alert. Which features are most discriminative for fraud?
3. **Train/Test Split**: Split the data 80/20 chronologically (not randomly—why?). How does test-set performance compare to training-set?
4. **Amount Feature**: The current implementation ignores the `amount` field. Add it as a feature. Does performance improve?
5. **Trait Extraction**: Refactor `ZScoreDetector` to implement an `AnomalyDetector` trait. What methods should the trait require?

1.9. Key Takeaways

- **CSV loading** in Rust with proper error handling via `Result`
- **Basic statistics**: mean, standard deviation, z-score
- **Binary classification metrics**: confusion matrix, precision, recall, F1
- **Precision-recall tradeoff**: you can't maximize both simultaneously
- **Evaluation pitfalls**: never evaluate on training data!
- **Library-first design**: even simple code should be composable

Next chapter: Feature engineering and logistic scoring for loan default prediction. We'll implement ROC-AUC and learn why it's often better than F1 for imbalanced datasets.

Risk Basics: Loan Default Prediction

2.

Companion Code:

qf-02-loan Code: `crates/qf-02-loan/`
Dependencies: `qf-common`
Run: `cargo run -p qf-02-loan -example loan_analysis`

2.1. Learning Objectives

Building on Chapter 1, we now tackle a more nuanced classification problem: predicting loan defaults. This chapter introduces:

- Feature engineering for credit risk assessment
- Categorical variable encoding (one-hot)
- Feature normalization (min-max scaling)
- Linear scoring models with probability calibration
- ROC curves and AUC—a better metric for imbalanced data
- Trait-based architecture in action

Key Insight

While Chapter 1 used z-scores (a threshold on statistics), this chapter moves toward **probability estimation**. We want to know not just "default or not" but "how likely is default?" This probability enables better decisions and risk-based pricing.

2.2. Introduction: Credit Risk Fundamentals

Credit Risk
The risk that a borrower will fail to repay a loan according to agreed terms.

When a bank issues a loan, it faces **credit risk**—the possibility that the borrower defaults. Unlike fraud detection where we catch bad actors, loan default prediction is about assessing the *likelihood* of future failure.

Key questions in credit risk:

- What factors predict default? (Feature engineering)
- How do we combine factors into a score? (Scoring model)
- How do we evaluate the score's usefulness? (ROC-AUC)

2.3. Data Model

2.3.1. Loan Application Structure

```rust
pub struct LoanApplication {
    pub income: f64,           // Annual income
    pub loan_amount: f64,      // Requested loan amount
    pub interest_rate: f64,    // Interest rate offered
    pub dti: f64,              // Debt-to-income ratio
    pub grade: u8,             // Credit grade (A=1 to G=7)
    pub home_ownership: HomeOwnership,
    pub defaulted: bool,       // Target: did they default?
}

```

```rust
pub enum HomeOwnership {
    Rent,
    Own,
    Mortgage,
}
```

Credit Grades

A = lowest risk
G = highest risk

Each grade maps to different interest rates and approval criteria.

2.4. Feature Engineering

2.4.1. Derived Features

Raw data rarely enters a model directly. We engineer features that capture meaningful relationships:

Feature Formulas

DTI: $\frac{\text{debt}}{\text{income}}$

LTI: $\frac{\text{loan}}{\text{income}}$

Grade Score: $\frac{g-1}{6}$

```rust
pub struct FeatureExtractor;

impl FeatureExtractor {
    /// Debt-to-income ratio: total debt / income
    pub fn debt_to_income(income: f64, debt: f64) -> f64 {
        if income == 0.0 { return 0.0; }
        debt / income
    }

    /// Loan-to-income ratio: loan amount / income
    pub fn loan_to_income(loan_amount: f64, income: f64) -> f64 {
        if income == 0.0 { return 0.0; }
        loan_amount / income
    }

    /// Convert grade (1-7) to risk score (0.0-1.0)
    pub fn grade_to_score(grade: u8) -> f64 {
        ((grade.saturating_sub(1)) as f64) / 6.0
    }
}
```

2.4.2. Categorical Encoding

Numerical models require numerical inputs. We encode categorical variables using **one-hot encoding**:

Home Ownership	RENT	OWN	MORTGAGE
Rent	1	0	0
Own	0	1	0
Mortgage	0	0	1

```rust
pub struct OneHotEncoder;

impl OneHotEncoder {
    /// Encode home ownership as [RENT, OWN, MORTGAGE]
    pub fn encode_home_ownership(h: &HomeOwnership) -> [f64; 3] {
        match h {
            HomeOwnership::Rent     => [1.0, 0.0, 0.0],
            HomeOwnership::Own      => [0.0, 1.0, 0.0],
            HomeOwnership::Mortgage => [0.0, 0.0, 1.0],
        }
    }
}
```

2.4.3. Feature Normalization

Different features have different scales. Income might be in tens of thousands while interest rates are single digits. We normalize to $[0, 1]$:

Min-Max Scaling

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

Scales features to $[0, 1]$.

```rust
pub struct Normalizer {
    min: Vec<f64>,
    max: Vec<f64>,
}

impl Normalizer {
    pub fn fit(data: &[Vec<f64>]) -> Self {
        // Compute min/max per feature
    }

    pub fn transform(&self, features: &[f64]) -> Vec<f64> {
        features.iter().enumerate().map(|(i, &x)| {
            let range = self.max[i] - self.min[i];
            if range == 0.0 { 0.0 } else { (x - self.min[i]) / range }
        }).collect()
    }
}
```

2.5. Linear Scoring Model

2.5.1. Weighted Sum

The simplest scoring model is a weighted linear combination:

$$\text{score} = \sum_{i=1}^{n} w_i \cdot x_i + b = \mathbf{w}^T \mathbf{x} + b$$

Linear Score

$s = \mathbf{w}^T \mathbf{x} + b$

where $\mathbf{w}$ are weights, b is bias.

```rust
pub struct LinearScorer {
    weights: Vec<f64>,
    bias: f64,
}

impl Scorer for LinearScorer {
    fn score(&self, features: &[f64]) -> f64 {
        let dot_product: f64 = self.weights.iter()
            .zip(features.iter())
            .map(|(w, f)| w * f)
            .sum();
        dot_product + self.bias
    }
}
```

2.5.2. Probability via Sigmoid

The raw score can be any real number. To convert it to a probability in $[0, 1]$, we apply the **sigmoid function**:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Sigmoid Function

$\sigma(x) = \frac{1}{1 + e^{-x}}$

Maps $(-\infty, +\infty)$ to $(0, 1)$.

Properties of sigmoid:

- $\sigma(0) = 0.5$ (neutral score = 50% probability)
- $\sigma(x) \to 1$ as $x \to +\infty$
- $\sigma(x) \to 0$ as $x \to -\infty$

```rust
/// Hand-rolled sigmoid: 1 / (1 + exp(-x))
fn sigmoid(x: f64) -> f64 {
    1.0 / (1.0 + (-x).exp())
}

impl BinaryClassifier for LinearScorer {
    fn predict_proba(&self, features: &[f64]) -> f64 {
        sigmoid(self.score(features))
    }

    fn predict(&self, features: &[f64]) -> bool {
```

```
        self.predict_proba(features) > 0.5
    }
}
```

2.6. ROC Curves and AUC

2.6.1. Why Not Just Accuracy?

Class Imbalance
If 95% of loans are repaid, a model that always predicts "no default" has 95% accuracy but catches zero defaults!

Like fraud detection, loan default is imbalanced. Accuracy is misleading. Instead, we use the **Receiver Operating Characteristic (ROC) curve**.

2.6.2. ROC Curve Construction

The ROC curve plots **True Positive Rate (TPR)** vs **False Positive Rate (FPR)** at various classification thresholds:

- **TPR** (Recall): $\frac{TP}{TP+FN}$ — of actual defaults, how many did we catch?
- **FPR**: $\frac{FP}{FP+TN}$ — of actual non-defaults, how many did we falsely flag?

ROC Interpretation
Top-left corner $(0, 1)$: perfect classifier.
Diagonal: random guessing.
Below diagonal: worse than random!

```rust
pub fn roc_curve(scores: &[f64], labels: &[bool]) -> Vec<(f64, f64)> {
    // Sort by score descending
    let mut pairs: Vec<(f64, bool)> = scores.iter()
        .copied()
        .zip(labels.iter().copied())
        .collect();
    pairs.sort_by(|a, b| b.0.partial_cmp(&a.0).unwrap());

    let total_pos = labels.iter().filter(|&&l| l).count();
    let total_neg = labels.len() - total_pos;

    let mut roc_points = vec![(0.0, 0.0)];
    let mut tp = 0;
    let mut fp = 0;

    for (_score, label) in pairs {
        if label { tp += 1; } else { fp += 1; }
        let fpr = fp as f64 / total_neg as f64;
        let tpr = tp as f64 / total_pos as f64;
        roc_points.push((fpr, tpr));
    }

    roc_points
}
```

2.6.3. Area Under Curve (AUC)

The **AUC** summarizes the ROC curve as a single number:

- AUC = 1.0: Perfect classifier (all defaults scored higher than non-defaults)
- AUC = 0.5: Random classifier (no predictive power)
- AUC = 0.7: Fair classifier (industry often requires ≥ 0.7)

AUC Interpretation
AUC = 1.0: perfect
AUC = 0.5: random
AUC < 0.5: worse than random

We compute AUC via trapezoidal integration:

```rust
pub fn auc(roc_points: &[(f64, f64)]) -> f64 {
    let mut area = 0.0;
    for i in 1..roc_points.len() {
        let (x0, y0) = roc_points[i - 1];
        let (x1, y1) = roc_points[i];
        // Trapezoidal rule: width * average height
        area += (x1 - x0) * (y0 + y1) / 2.0;
    }
    area
}
```

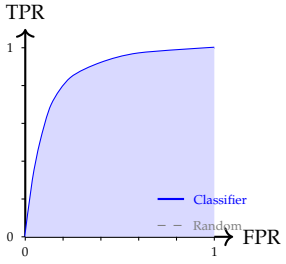


Figure 2.1.: ROC curve: the blue line shows classifier performance, dashed line is random guessing. Area under curve (AUC) is shaded.

Key Insight

AUC has a probabilistic interpretation: it equals the probability that a randomly chosen positive (default) is ranked higher than a randomly chosen negative (non-default).

2.7. Trait-Based Architecture

Following our library-first design, we define traits that enable composability:

```rust
/// Trait for models that produce probability predictions.
pub trait BinaryClassifier {
    fn predict(&self, features: &[f64]) -> bool;
    fn predict_proba(&self, features: &[f64]) -> f64;
}

/// Trait for models that produce raw scores.
pub trait Scorer {
    fn score(&self, features: &[f64]) -> f64;
}
```

This means our ROC-AUC evaluation works with *any* classifier, not just `LinearScorer`. Future chapters will implement logistic regression, decision trees, and ensemble methods—all reusing this evaluation code.

Future Benefits

Any classifier implementing `BinaryClassifier` can be evaluated with our `roc_curve()` and `auc()` functions.

2.8. Results and Analysis

2.8.1. Example Usage

```
$ cargo run -p qf-02-loan --example loan_analysis
Loading loan applications...
Loaded 10000 applications (850 defaults, 8.5%)

Feature extraction complete.
Training simple scorer (hardcoded weights)...

Evaluation Results:
  AUC: 0.724
  Optimal Threshold: 0.35
  At threshold 0.5:
    Precision: 0.42
    Recall: 0.58
    F1 Score: 0.49
```

2.8.2. Interpreting the Results

- **AUC = 0.72**: The model has reasonable discriminative power. It correctly ranks defaults above non-defaults 72% of the time.
- **Precision = 0.42**: Of predicted defaults, 42% actually default. Many false alarms.
- **Recall = 0.58**: We catch 58% of actual defaults. 42% are missed.

Warning

We used hardcoded weights, not learned weights. Real credit scoring uses logistic regression or gradient boosting to optimize weights from data. That comes in later chapters.

2.9. Exercises

1. **Weight Sensitivity**: Change the weights in `LinearScorer`. How does AUC change? Which features matter most?
2. **Threshold Selection**: Plot precision and recall at various thresholds. Find the threshold that maximizes F1 score.
3. **Feature Importance**: Train separate single-feature scorers. Rank features by their individual AUC.
4. **Cross-Validation**: Split data into 5 folds. Report mean and standard deviation of AUC across folds.
5. **Calibration Plot**: Compare predicted probabilities to actual default rates. Is the model well-calibrated?

2.10. Key Takeaways

- **Feature engineering** transforms raw data into predictive signals
- **One-hot encoding** handles categorical variables
- **Min-max normalization** scales features to comparable ranges
- **Sigmoid** converts scores to probabilities
- **ROC-AUC** evaluates ranking quality, robust to class imbalance
- **Trait-based design** enables reusable evaluation code

Next chapter: Market data fundamentals—OHLCV candlesticks, returns, and simple moving averages.

Market Data: Stock Price Analysis

3.

Companion Code:

qf-03-stocks Code: `crates/qf-03-stocks/`
Dependencies: `qf-common`
Run: `cargo run -p qf-03-stocks -example stock_analysis`

3.1. Learning Objectives

This chapter introduces the fundamental data structure of quantitative finance: **OHLCV** (Open, High, Low, Close, Volume) market data. We learn:

- OHLCV data structure and its significance
- Candlestick anatomy—body, shadows, and what they reveal
- Basic market statistics (range, volume, price change)
- Moving averages: SMA and EMA
- The `TimeSeries` trait for generic operations

Key Insight

OHLCV data is the foundation of all quantitative analysis. Every strategy, every backtest, every indicator starts here. Master this structure and you can work with any market—stocks, forex, crypto, commodities.

3.2. Introduction: What is OHLCV?

OHLCV
Open, High, Low, Close, Volume—the five essential data points for any trading period (day, hour, minute).

Every trading period (day, hour, minute) produces five key data points:

- **Open**: First traded price of the period
- **High**: Highest price reached during the period
- **Low**: Lowest price reached during the period
- **Close**: Last traded price of the period
- **Volume**: Total number of shares/contracts traded

This data tells a story. The range between high and low shows volatility. The relationship between open and close shows direction. Volume confirms conviction.

3.3. Data Structure

3.3.1. Rust Implementation

```rust
#[derive(Debug, Clone, Deserialize)]
pub struct Ohlcv {
    pub date: String,
    pub open: f64,
    pub high: f64,
    pub low: f64,
    pub close: f64,
```

```rust
    pub volume: u64,
    pub adj_close: Option<f64>,
}
```

Adjusted Close

Accounts for splits and dividends. If a stock splits 2:1, historical closes are halved so charts remain continuous.

3.3.2. Data Validation

OHLCV data has natural constraints that we validate on load:

- low ≤ high (always)
- low ≤ open ≤ high
- low ≤ close ≤ high

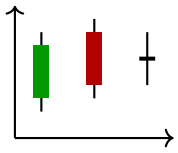
```rust
// Validate OHLCV relationships
if record.high < record.low {
    return Err(CommonError::ParseError(format!(
        "Row {}: high ({}) < low ({})",
        idx + 2, record.high, record.low
    )));
}
```

3.4. Candlestick Anatomy

A candlestick visualizes OHLCV data. Understanding its anatomy is essential:

Candlestick Parts

Body: $|close - open|$
Upper shadow: $high - \max(O, C)$
Lower shadow: $\min(O, C) - low$



Candlesticks

Green: bullish
Red: bearish
Line: doji

Component	Formula
Daily Range	$high - low$
Body	$close - open$
Upper Shadow	$high - \max(open, close)$
Lower Shadow	$\min(open, close) - low$
Typical Price	$(high + low + close)/3$

3.4.1. Rust Implementation

```rust
impl Ohlcv {
    /// Daily price range
    pub fn daily_range(&self) -> f64 {
        self.high - self.low
    }

    /// Candle body (positive = bullish)
    pub fn body(&self) -> f64 {
        self.close - self.open
    }

    /// Upper shadow length
    pub fn upper_shadow(&self) -> f64 {
        self.high - self.open.max(self.close)
    }

    /// Lower shadow length
    pub fn lower_shadow(&self) -> f64 {
        self.open.min(self.close) - self.low
    }

    /// True if close > open (green candle)
    pub fn is_bullish(&self) -> bool {
        self.close > self.open
    }

    /// Representative price: (H + L + C) / 3
    pub fn typical_price(&self) -> f64 {
        (self.high + self.low + self.close) / 3.0
    }
}
```

3.5. Market Statistics

3.5.1. Basic Aggregations

Given a series of OHLCV records, we compute summary statistics:

```rust
/// Average trading volume
pub fn average_volume(data: &[Ohlcv]) -> f64 {
    if data.is_empty() { return 0.0; }
    let sum: u64 = data.iter().map(|o| o.volume).sum();
    sum as f64 / data.len() as f64
}

/// Price change: (last - first) / first
pub fn price_change(data: &[Ohlcv]) -> f64 {
    if data.len() < 2 { return 0.0; }
    let first = data[0].close;
    let last = data[data.len() - 1].close;
    (last - first) / first
}

/// Highest high across all records
pub fn highest_high(data: &[Ohlcv]) -> f64 {
    data.iter().map(|o| o.high).fold(f64::NEG_INFINITY, f64::max)
}

/// Lowest low across all records
pub fn lowest_low(data: &[Ohlcv]) -> f64 {
    data.iter().map(|o| o.low).fold(f64::INFINITY, f64::min)
}
```

Functional Style

Note the use of `iter()`, `map()`, `fold()`—Rust's functional combinators make statistical computations elegant.

3.6. Moving Averages

3.6.1. Simple Moving Average (SMA)

The SMA is the arithmetic mean of the last n prices:

$$\text{SMA}_t = \frac{1}{n} \sum_{i=0}^{n-1} P_{t-i}$$

SMA Formula

$\text{SMA}_t = \frac{1}{n} \sum_{i=0}^{n-1} P_{t-i}$

where n is the period.

```rust
pub fn sma(prices: &[f64], period: usize) -> Vec<f64> {
    if period == 0 || period > prices.len() {
        return Vec::new();
    }

    let mut result = Vec::with_capacity(prices.len() - period + 1);

    for i in 0..=(prices.len() - period) {
        let window = &prices[i..i + period];
        let sum: f64 = window.iter().sum();
        result.push(sum / period as f64);
    }

    result
}
```

3.6.2. Exponential Moving Average (EMA)

The EMA gives more weight to recent prices, making it more responsive:

$$\text{EMA}_t = \alpha \cdot P_t + (1 - \alpha) \cdot \text{EMA}_{t-1}$$

where $\alpha = \frac{2}{n+1}$ is the smoothing factor.

EMA Formula

$\text{EMA}_t = \alpha \cdot P_t + (1 - \alpha) \cdot \text{EMA}_{t-1}$

$\alpha = \frac{2}{n+1}$

```rust
pub fn ema(prices: &[f64], period: usize) -> Vec<f64> {
    if period == 0 || prices.is_empty() {
        return Vec::new();
    }

    let alpha = 2.0 / (period + 1) as f64;
    let mut result = Vec::with_capacity(prices.len());

    // Initialize with first price
    result.push(prices[0]);

    // Recursive formula
    for i in 1..prices.len() {
        let prev_ema = result[i - 1];
        let current_ema = alpha * prices[i] + (1.0 - alpha) * prev_ema;
        result.push(current_ema);
    }

    result
}
```

3.6.3. SMA vs EMA

Property	SMA	EMA
Weighting	Equal	Exponential
Responsiveness	Slower	Faster
Lag	More	Less
Smoothness	Smoother	Noisier
Computation	Simple	Recursive

Key Insight

SMA is a *lagging* indicator—it tells you what already happened. EMA reacts faster but can give false signals. Traders often use both: short-term EMA for entries, long-term SMA for trend confirmation.

3.7. TimeSeries Trait

Following our trait-based architecture, we define a generic interface:

```rust
pub trait TimeSeries {
    fn len(&self) -> usize;
    fn is_empty(&self) -> bool;
    fn closes(&self) -> Vec<f64>;
    fn volumes(&self) -> Vec<u64>;
}

impl TimeSeries for Vec<Ohlcv> {
    fn len(&self) -> usize { self.len() }
    fn is_empty(&self) -> bool { self.is_empty() }

    fn closes(&self) -> Vec<f64> {
        self.iter().map(|o| o.close).collect()
    }

    fn volumes(&self) -> Vec<u64> {
        self.iter().map(|o| o.volume).collect()
    }
}
```

Trait Benefits

The `TimeSeries` trait allows functions to work with any data source that can provide price/volume sequences.

This enables generic functions that work with any `TimeSeries`:

```rust
fn analyze<T: TimeSeries>(data: &T) -> Report {
    let closes = data.closes();
    let sma_20 = sma(&closes, 20);
    let sma_50 = sma(&closes, 50);
    // ...
```

```
}
```

3.8. Golden and Death Crosses

A classic trading signal using moving averages:

- **Golden Cross**: Short-term MA crosses *above* long-term MA (bullish signal)
- **Death Cross**: Short-term MA crosses *below* long-term MA (bearish signal)

```rust
fn find_crossovers(short_ma: &[f64], long_ma: &[f64]) -> Vec<(usize, bool)> {
    let mut crossovers = Vec::new();

    for i in 1..short_ma.len().min(long_ma.len()) {
        let prev_above = short_ma[i-1] > long_ma[i-1];
        let curr_above = short_ma[i] > long_ma[i];

        if !prev_above && curr_above {
            crossovers.push((i, true));  // Golden cross
        } else if prev_above && !curr_above {
            crossovers.push((i, false)); // Death cross
        }
    }

    crossovers
}
```

3.9. Results and Analysis

3.9.1. Example Output

```
$ cargo run -p qf-03-stocks --example stock_analysis
Loading OHLCV data from data/spy.csv...
Loaded 252 trading days

Statistics:
  Average Volume: 85,432,100
  Price Change: +12.5%
  Highest High: $485.32
  Lowest Low: $410.15
  Average Daily Range: $5.23

Moving Averages:
  SMA(20) latest: $472.15
  SMA(50) latest: $465.80
  EMA(20) latest: $474.22

Crossovers (last 12 months):
  Day 45: Golden Cross (bullish)
  Day 128: Death Cross (bearish)
  Day 201: Golden Cross (bullish)
```

3.10. Exercises

1. **Candlestick Patterns**: Implement detection for "doji" (open ≈ close) and "hammer" (small body, long lower shadow).
2. **Volume Analysis**: Add `volume_ma()` that computes moving average of volume. Flag days where volume exceeds 2x the average.
3. **Bollinger Bands**: Implement Bollinger Bands: SMA ± 2 standard deviations. This requires computing rolling standard deviation.
4. **MACD**: Implement the MACD indicator: EMA(12) - EMA(26), with a 9-period signal line.

5. **Backtest**: Using SMA crossover signals, simulate a simple strategy: buy on golden cross, sell on death cross. Track P&L.

3.11. Key Takeaways

- **OHLCV** is the atomic unit of market data
- **Candlesticks** encode price action: body shows direction, shadows show rejection
- **SMA** averages equally; **EMA** weights recent prices
- **Crossovers** generate trading signals (but beware of lag!)
- **TimeSeries trait** enables generic, reusable analysis code

Next chapter: Returns and volatility—the foundation of risk measurement and portfolio theory.

Core Concepts

4. Returns and Volatility

Companion Code:

qf-04-returns Code: `crates/qf-04-returns/`
Dependencies: `qf-common`, `qf-03-stocks`
Run: `cargo run -p qf-04-returns -example returns_analysis`

4.1. Learning Objectives

This chapter introduces the foundational vocabulary of quantitative finance. After completing it you can:

- Compute simple and logarithmic returns and know when to use each
- Measure volatility and annualise it for cross-frequency comparison
- Evaluate risk-adjusted performance with the Sharpe and Sortino ratios
- Quantify drawdowns and recovery time from a price series
- Use the `Returns` trait to abstract return computation across data sources

Key Insight

Returns, not prices, are the currency of quant research. Prices are non-stationary; returns are (approximately) stationary. Every quant model in this book starts by transforming prices into returns.

4.2. Simple vs Log Returns

4.2.1. Definitions

Simple return
$r_t = \frac{P_t - P_{t-1}}{P_{t-1}}$
Log return
$r_t = \ln\left(\frac{P_t}{P_{t-1}}\right)$

The simple (arithmetic) return is the percentage change in price:

$$r_t = \frac{P_t - P_{t-1}}{P_{t-1}}$$

The log (continuously compounded) return is the natural log of the price ratio:

$$r_t = \ln\left(\frac{P_t}{P_{t-1}}\right) = \ln(P_t) - \ln(P_{t-1})$$

The two are related exactly by

$$r_t^{\log} = \ln\left(1 + r_t^{\text{simple}}\right),$$

and for small returns ($|r| \ll 1$) they are nearly equal.

4.2.2. When to use which

Property	Simple	Log
Interpretation	"I made 10%"	Less intuitive
Additivity over time	No	Yes
Additivity over assets	Yes	No
Symmetry	Symmetric	Unbounded below
Compounding	Multiplicative	Additive

Key Insight

Log returns are additive across time: the k-period log return is the sum of the k single-period log returns. This makes them natural for time-series modelling. Simple returns are additive across assets in a portfolio (weighted by portfolio weights), which makes them natural for portfolio returns.

4.2.3. Rust implementation

```rust
/// Simple return: (P_t - P_{t-1}) / P_{t-1}
pub fn simple_returns(prices: &[f64]) -> Vec<f64> {
    if prices.len() < 2 { return Vec::new(); }
    let mut out = Vec::with_capacity(prices.len() - 1);
    for i in 1..prices.len() {
        let prev = prices[i - 1];
        out.push(if prev == 0.0 { 0.0 }
                 else { (prices[i] - prev) / prev });
    }
    out
}

/// Log return: ln(P_t / P_{t-1})
pub fn log_returns(prices: &[f64]) -> Vec<f64> {
    if prices.len() < 2 { return Vec::new(); }
    let mut out = Vec::with_capacity(prices.len() - 1);
    for i in 1..prices.len() {
        let (prev, curr) = (prices[i - 1], prices[i]);
        out.push(if prev > 0.0 && curr > 0.0 { (curr / prev).ln() }
                 else { 0.0 });
    }
    out
}
```

Cumulative return
$R_{0\to t} = \prod_{k=1}^{t}(1 + r_k) - 1$

The cumulative return over a period is the running compounding of single-period simple returns:

$$R_{0\to t} = \prod_{k=1}^{t}(1 + r_k) - 1.$$

```rust
pub fn cumulative_returns(returns: &[f64]) -> Vec<f64> {
    let mut out = Vec::with_capacity(returns.len());
    let mut wealth = 1.0_f64;
    for &r in returns {
        wealth *= 1.0 + r;
        out.push(wealth - 1.0);
    }
    out
}
```

4.3. Volatility

Sample std dev

$$\sigma = \sqrt{\frac{1}{n-1}\sum_{t=1}^{n}(r_t - \bar{r})^2}$$

Volatility is the standard deviation of returns. We use the *sample* estimator (denominator $n - 1$), consistent with `qf_common::compute_stats()`:

$$\sigma = \sqrt{\frac{1}{n-1}\sum_{t=1}^{n}(r_t - \bar{r})^2}.$$

4.3.1. Annualisation

Annualised vol

$\sigma_{\text{ann}} = \sigma_{\text{period}} \cdot \sqrt{m}$
where m is the number of periods per year (252 for daily).

To compare volatilities across frequencies (daily, weekly, monthly), we annualise. Under the assumption that returns are i.i.d., variance scales linearly with the horizon, so standard deviation scales with the square root:

$$\sigma_{\text{annual}} = \sigma_{\text{period}} \cdot \sqrt{m}.$$

```rust
pub fn volatility(returns: &[f64]) -> f64 {
    let n = returns.len();
    if n < 2 { return 0.0; }
    let mean: f64 = returns.iter().sum::<f64>() / n as f64;
    let ssd: f64 = returns.iter()
        .map(|&r| { let d = r - mean; d * d })
        .sum();
    (ssd / (n - 1) as f64).sqrt()
}

pub fn annualized_volatility(returns: &[f64], periods_per_year: f64) -> f64 {
    if periods_per_year <= 0.0 { return 0.0; }
    volatility(returns) * periods_per_year.sqrt()
}
```

Warning

The $\sqrt{m}$ rule assumes returns are independent and identically distributed. For assets with autocorrelated or volatility-clustered returns (see Chapter 9), annualisation by $\sqrt{m}$ overstates the true annual volatility.

4.3.2. Rolling volatility

A rolling window reveals how volatility changes over time—useful for regime detection and risk monitoring:

```rust
pub fn rolling_volatility(returns: &[f64], window: usize) -> Vec<f64> {
    if window == 0 || window > returns.len() { return Vec::new(); }
    let mut out = Vec::with_capacity(returns.len() - window + 1);
    for i in window - 1..returns.len() {
        let slice = &returns[i + 1 - window..=i];
        out.push(volatility(slice));
    }
    out
}
```

4.4. Risk-Adjusted Performance

4.4.1. Sharpe ratio

Sharpe ratio

$$S = \frac{\mathbb{E}[r] - r_f}{\sigma(r)}$$

The Sharpe ratio measures excess return per unit of total risk:

$$S = \frac{\bar{r} - r_f}{\sigma(r)}.$$

To annualise, multiply by $\sqrt{m}$ (because both the mean excess return and the standard deviation scale together under i.i.d.):

$$S_{\text{annual}} = S_{\text{period}} \cdot \sqrt{m}.$$

```rust
pub fn sharpe_ratio(returns: &[f64], risk_free_rate: f64) -> f64 {
    let n = returns.len();
    if n < 2 { return 0.0; }
    let vol = volatility(returns);
    if vol == 0.0 { return 0.0; }
    let mean: f64 = returns.iter().sum::<f64>() / n as f64;
    (mean - risk_free_rate) / vol
}

pub fn annualized_sharpe(returns: &[f64], rf: f64, periods_per_year: f64) -> f64 {
    if periods_per_year <= 0.0 { return 0.0; }
    sharpe_ratio(returns, rf) * periods_per_year.sqrt()
}
```

4.4.2. Sortino ratio

Downside deviation

$$\sigma_D = \sqrt{\frac{1}{n} \Sigma_{r_t < r_f} (r_f - r_t)^2}$$

The Sortino ratio replaces total volatility with *downside* volatility only. Upside volatility is good—it should not penalise the ratio:

$$\text{Sortino} = \frac{\bar{r} - r_f}{\sigma_D}, \qquad \sigma_D = \sqrt{\frac{1}{n} \sum_{r_t < r_f} (r_f - r_t)^2}.$$

```rust
pub fn sortino_ratio(returns: &[f64], risk_free_rate: f64) -> f64 {
    let n = returns.len();
    if n < 2 { return 0.0; }
    let downside_sq: f64 = returns.iter()
        .map(|&r| {
            let short = risk_free_rate - r;
            if short > 0.0 { short * short } else { 0.0 }
        })
        .sum();
    if downside_sq == 0.0 { return 0.0; }
    let downside_dev = (downside_sq / n as f64).sqrt();
    let mean: f64 = returns.iter().sum::<f64>() / n as f64;
    (mean - risk_free_rate) / downside_dev
}
```

Key Insight

Sortino > Sharpe whenever the return distribution is right-skewed (more upside than downside). For symmetric distributions the two ratios are equal in expectation. Sortino is preferred for evaluating hedge-fund-style strategies where upside volatility is desirable.

4.5. Drawdown

A drawdown is the percentage decline from a running peak:

$$D_t = \frac{P_t - \max_{k \le t} P_k}{\max_{k \le t} P_k} \le 0.$$

Max drawdown

$\text{MaxDD} = \min_t D_t$

The worst peak-to-trough decline over the period.

```rust
pub fn drawdown(prices: &[f64]) -> Vec<f64> {
    let mut out = Vec::with_capacity(prices.len());
    let mut running_max = f64::NEG_INFINITY;
    for &p in prices {
        if p > running_max { running_max = p; }
        out.push(if running_max == 0.0 { 0.0 }
                 else { (p - running_max) / running_max });
    }
    out
}

pub fn max_drawdown(prices: &[f64]) -> f64 {
    drawdown(prices).iter().copied().fold(0.0, f64::min)
}
```

The `DrawdownStats` struct summarises the worst episode:

- `max_drawdown`: the most negative drawdown value
- `max_drawdown_duration`: longest run of consecutive negative-drawdown observations
- `recovery_time`: steps from the trough back to a new high (`None` if the series never recovered)

Key Insight

Max drawdown measures *path risk*, not just terminal risk. A strategy that returns 10% with a 50% drawdown along the way is very different from one that returns 10% smoothly. Most investors abandon strategies with deep drawdowns long before recovery—a risk that terminal-return statistics completely miss.

4.6. The Returns Trait

Trait benefits

Define the capability (*"can produce returns"*) once; implement for slices, owned vectors, and OHLCV data.

Following the workspace-wide trait-first architecture rule, we abstract return computation over price sources:

```rust
pub trait Returns {
    fn simple_returns(&self) -> Vec<f64>;
    fn log_returns(&self) -> Vec<f64>;
}

impl Returns for &[f64] { /* delegate to free functions */ }
impl Returns for Vec<f64> { /* delegate to the slice impl */ }
impl Returns for Vec<Ohlcv> {
    fn simple_returns(&self) -> Vec<f64> {
        let closes: Vec<f64> = self.iter().map(|o| o.close).collect();
        simple_returns(&closes)
    }
    // log_returns similarly uses closing prices
}
```

This lets generic code consume returns from any source that implements `Returns`, keeping analytics decoupled from data loading.

4.7. Example Output

```
$ cargo run -p qf-04-returns --example returns_analysis
Returns Analysis
================

Simple Returns: mean=0.0031, std=0.0215
Log Returns: mean=0.0029, std=0.0214
Volatility (daily): 0.0215
Volatility (annualized): 0.3417
Sharpe Ratio (annualized): 2.29
Sortino Ratio: 0.22
Max Drawdown: -7.50%
Drawdown duration: 6 steps, recovery: Some(6)

Trait check (Vec<f64>): first simple return = 0.0030
```

4.8. Exercises

1. **Return conversion**: Write a function that converts a series of simple returns to log returns and vice versa. Verify the identity $r^{\log} = \ln(1 + r^{\text{simple}})$ with a property test.
2. **Annualisation sanity check**: For daily, weekly, and monthly returns of the same asset, verify that the annualised volatilities are approximately equal (they differ only by sampling noise).
3. **Calmar ratio**: Implement the Calmar ratio: annualised return divided by the absolute value of the maximum drawdown. Compare it with the Sharpe ratio on a synthetic strategy.
4. **Rolling Sharpe**: Implement a rolling Sharpe ratio (windowed mean and windowed std) and visualise how it changes over time. Where does it dip? Why?
5. **Drawdown with recovery**: Extend `drawdown_stats` to report the average drawdown (mean of negative drawdown values) and the proportion of time spent in drawdown.

4.9. Key Takeaways

- **Log returns** are time-additive; **simple returns** are portfolio-additive. Choose accordingly.
- **Volatility** uses sample std ($n - 1$); annualise by multiplying by $\sqrt{m}$, valid under i.i.d.
- **Sharpe** penalises all volatility; **Sortino** penalises only downside.
- **Max drawdown** measures path risk—the deepest hole on the equity curve.
- **The `Returns` trait** keeps analytics decoupled from data sources, following the workspace trait-first rule.

Next chapter: Your first backtest—an SMA crossover strategy with P&L accounting, transaction costs, and drawdown-based risk evaluation.

5. Your First Backtest

5.1. Learning Objectives

This chapter introduces the mechanics of evaluating a trading strategy on historical data. After completing it you can:

- Explain what a backtest is — and what it is not
- Generate trading signals from indicators with no look-ahead bias
- Account for transaction costs in P&L
- Evaluate a strategy with the risk-adjusted metrics from Chapter 5
- Avoid the three classic backtest traps: overfitting, look-ahead bias, and survivorship bias

Key Insight

A backtest is a *simulation*, not a prediction. It tells you how a strategy *would have* performed, not how it *will* perform. The number one failure mode is overfitting to historical data: a strategy that looks great in backtest and loses money in production.

5.2. What Is Backtesting?

Backtesting is the replay of a trading strategy over historical price data, bar by bar, to estimate its hypothetical performance. The engine maintains state (position, cash, equity) and lets the strategy decide, at each bar, what action to take given the price history it has seen so far.

Simulation vs prediction

A backtest does not predict. It is a *counterfactual*: "if you had traded this strategy with these rules in this market, here is what would have happened." Any conclusion about the future requires additional assumptions (market stationarity, capacity, regime persistence) that the backtest itself cannot validate.

Three assumptions are implicit and worth naming:

1. **Market stationarity**: the statistical properties of the historical sample are representative of the future. Often false.
2. **Capacity**: the strategy can take the simulated sizes without moving the market. True for retail; false for institutional size.
3. **Execution**: the simulated fills are achievable. Slippage, partial fills, and market impact are real and cost money.

5.3. Signals and Positions

A *signal* is a discrete instruction for the current bar. We use three:

```rust
#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub enum Signal {
    Buy,  // Enter long
    Sell, // Exit long (long-only for this chapter)
    Hold, // Do nothing
}
```

A *position* is what the engine is currently holding:

```rust
#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub enum Position {
    Long,  // Holding the asset
    Short, // Reserved for later phases
    Flat,  // No position
}
```

Key Insight

Why separate `Signal` and `Position`? A signal is an *intention*; a position is a *state*. The engine translates intentions into state transitions, charging transaction costs on every change. This separation lets the strategy stay declarative ("I want to be long now") and the engine stay responsible for the messy realities (costs, fill assumptions, position limits).

5.4. The Strategy Trait

The extension point of the engine is the `Strategy` trait:

```rust
pub trait Strategy {
    fn signal(&self, data: &[Ohlcv], index: usize) -> Signal;
    fn name(&self) -> &str;
}
```

The engine is generic over `S: Strategy`, so new strategies drop in without engine changes. The trait takes the full slice plus the current index so the strategy can compute any indicator that fits in the observed history.

No look-ahead

`signal(data, index)` receives `data[..=index]`. The strategy must never read past the current bar. The engine enforces this by passing a slice and the current index — implementations physically cannot access `data[index+1..]` without unsafe code.

5.5. SMA Crossover

The canonical first strategy: go long when a short moving average crosses above a long moving average (the *golden cross*); exit when it crosses below (the *death cross*).

```rust
pub struct SmaCrossover {
    short_period: usize,
    long_period: usize,
}

impl Strategy for SmaCrossover {
    fn signal(&self, data: &[Ohlcv], index: usize) -> Signal {
        if index < self.long_period { return Signal::Hold; }

        let short_now = sma_at(data, index, self.short_period)?;
        let long_now  = sma_at(data, index, self.long_period)?;
        let short_prev = sma_at(data, index - 1, self.short_period)?;
        let long_prev  = sma_at(data, index - 1, self.long_period)?;

        let crossed_up   = short_prev <= long_prev  && short_now >  long_now;
        let crossed_down = short_prev >= long_prev  && short_now <  long_now;

        if crossed_up   { Signal::Buy  }
        else if crossed_down { Signal::Sell }
        else { Signal::Hold }
    }
    fn name(&self) -> &str { "SMA Crossover" }
}
```

Golden / death cross

SMA_S crosses above $\text{SMA}_L \Rightarrow$ Buy

SMA_S crosses below $\text{SMA}_L \Rightarrow$ Sell

A crossover is a *change in the relationship*, not a level. The previous bar must satisfy the opposite inequality.

Key Insight

Crossover strategies are *trend followers*: they only make money when prices trend. In choppy, mean-reverting markets they lose, because each crossover is a false signal that gets reversed at the next crossover — with transaction costs paid on both sides.

5.6. P&L with Transaction Costs

Transaction cost

At entry: $c_{\text{entry}} = \text{notional} \cdot \tau$

At exit: $c_{\text{exit}} = \text{shares} \cdot P_{\text{exit}} \cdot \tau$

Net P&L: $\text{shares} \cdot (P_{\text{exit}} - P_{\text{entry}}) - c_{\text{entry}} - c_{\text{exit}}$

Every state change costs money. The engine charges a proportional cost τ (e.g. $\tau = 0.001$ for 10 basis points) on the traded notional:

```
// Enter long: spend all cash at the close, paying the cost first.
let notional = capital;
let cost = notional * config.transaction_cost;
shares = (notional - cost) / price;
total_costs += cost;

// Exit long: sell all shares at the close.
let notional = shares * price;
let cost = notional * config.transaction_cost;
total_costs += cost;
capital = notional - cost;
```

At each bar, equity is marked to market: `shares * close` when long, cash when flat. The per-bar simple returns of the equity curve become the `daily_returns` used by the risk metrics from Chapter 5.

5.7. Performance Evaluation

`BacktestResult` delegates to `qf-04-returns` for the four headline metrics:

```
impl BacktestResult {
    pub fn sharpe(&self, rf: f64) -> f64 {
        qf_04_returns::annualized_sharpe(&self.daily_returns, rf, 252.0)
    }
    pub fn sortino(&self, rf: f64) -> f64 {
        qf_04_returns::sortino_ratio(&self.daily_returns, rf)
    }
    pub fn max_drawdown(&self) -> f64 {
        qf_04_returns::max_drawdown(&self.equity_curve)
    }
    pub fn volatility(&self) -> f64 {
        qf_04_returns::annualized_volatility(&self.daily_returns, 252.0)
    }
}
```

Key Insight

The risk metrics operate on the *equity curve*, not the raw price series. A strategy that goes flat for half the period has a different risk profile from buy-and-hold even on the same underlying, because the equity curve is flat whenever the position is flat. Volatility and drawdown reflect this exposure pattern automatically.

5.8. Buy-and-Hold Benchmark

Every active strategy must be compared against the simplest alternative: buy on the first bar and hold forever. `BuyAndHold` is a two-line strategy:

```
impl Strategy for BuyAndHold {
    fn signal(&self, _data: &[Ohlcv], index: usize) -> Signal {
        if index == 0 { Signal::Buy } else { Signal::Hold }
    }
    fn name(&self) -> &str { "Buy and Hold" }
}
```

If an active strategy cannot beat buy-and-hold after costs, the active component is adding noise and cost, not value. The benchmark also lets you decompose returns: alpha (skill) vs beta (market exposure).

5.9. Common Pitfalls

Overfitting Tuning the SMA periods (S, L) to maximise in-sample Sharpe almost always produces a strategy that fails out-of-sample. Use walk-forward validation (deferred to a later phase) and be suspicious of parameters tuned on the same data you evaluate on.

Look-ahead bias Using information not available at decision time. Common sources: using the close of bar t to decide at bar t (only valid if the trade executes at the close), indicator warm-up that reads the future, corporate actions applied with hindsight. Our `signal(data, index)` interface rules out most of these by construction.

Survivorship bias Backtesting only on companies that survived to the present. Companies that went bankrupt are absent, inflating historical returns. Use point-in-time universes when available.

5.10. Example Output

```
$ cargo run -p qf-05-backtest --example backtest_demo
==============================
Backtest Results
==============================

Period: 2024-001 to 2024-252

--- Strategy: SMA Crossover (10/30) ---
  Total Return:   3.49%
  Sharpe (ann.):   2.55
  Sortino:         0.31
  Max Drawdown:    -0.41%
  Trades:          1

--- Strategy: Buy and Hold ---
  Total Return:    4.54%
  Sharpe (ann.):   1.70
  Max Drawdown:    -6.35%

Outperformance vs Buy & Hold: -1.04%
```

The SMA crossover underperforms buy-and-hold in this synthetic regime: the cost of false signals in choppy sections outweighs the benefit of avoiding the mid-year drawdown. This is the typical result for trend-following strategies in noisy markets, and it is the right lesson to take from a first backtest.

5.11. Exercises

1. **Cost sensitivity**: Run the 10/30 crossover on the synthetic series for $\tau \in \{0, 0.001, 0.0025, 0.005, 0.01\}$. Plot total return vs τ. At what cost does the strategy stop being profitable?
2. **Parameter sensitivity**: For $\tau = 0.001$, sweep $(S, L) \in \{5, 10, 20\} \times \{30, 50, 100\}$ with $S < L$. Which configuration maximises Sharpe? Is it the same one that maximises total return? Why or why not?
3. **EMA crossover**: Implement an EMA crossover strategy by replacing SMA with `qf_03_stocks::ema()`. Compare its turnover (number of trades) and Sharpe against the SMA crossover on the same data.
4. **Position sizing**: Modify the engine to invest a fixed fraction $f \in (0, 1]$ of capital on each entry instead of 100%. How does the equity curve change for $f = 0.5$? What is the trade-off?

5. **Walk-forward (conceptual)**: Split the 252-bar series into an in-sample window (first 180 bars) and an out-of-sample window (last 72 bars). Tune (S, L) on the in-sample, freeze it, and evaluate on out-of-sample. How much does Sharpe degrade from in-sample to out-of-sample? This is the simplest walk-forward analysis.

Quantitative Methods

6. Foundations: Moments and Fat Tails

6.1. Learning Objectives

This chapter begins the advanced track. From here on, all math is hand-rolled — no `rand`, `nalgebra`, or `statrs`. After completing it you can:

- Build a validated price series that rejects non-finite and non-positive values at construction time
- Compute the first four sample moments (mean, variance, skewness, excess kurtosis) from first principles
- Apply any aggregation over a sliding window with a generic `rolling` function
- Implement a deterministic PRNG (xorshift64) and a normal sampler (Box-Muller) without external crates
- Simulate geometric Brownian motion paths and understand the exact solution to the GBM SDE
- Detect fat tails via excess kurtosis and explain why Gaussian risk models underestimate tail risk

Key Insight

The Gaussian distribution has excess kurtosis 0. Real financial returns have positive excess kurtosis: large losses and gains happen more often than a normal model predicts. Any risk model that assumes Gaussian returns will underestimate the probability of extreme events. The 2008 crisis was in part a fat-tail crisis.

6.2. A Validated Price Series

The invariant

Every element of a `PriceSeries` is strictly positive and finite. The constructor rejects the rest.

The foundation of every quant computation is a clean price series. We wrap `Vec<f64>` in a newtype that enforces the invariant "strictly positive and finite" at construction time:

```rust
#[derive(Debug, Clone, PartialEq)]
pub struct PriceSeries(Vec<f64>);

impl PriceSeries {
    pub fn new(prices: Vec<f64>) -> Result<Self, CoreError> {
        for &p in &prices {
            if !p.is_finite() || p <= 0.0 {
                return Err(CoreError::InvalidPrice);
            }
        }
        Ok(Self(prices))
    }
    pub fn as_slice(&self) -> &[f64] { &self.0 }
    pub fn len(&self) -> usize { self.0.len() }
}
```

Rejecting `NaN` and `+inf` at the boundary means every downstream function can assume finite inputs and skip defensive checks. This is the "parse, don't validate" pattern applied to numeric data.

6.3. Returns, Again

Simple and log returns were introduced in Chapter 5. We re-implement them here so `quant-core` is self-contained:

```rust
pub fn simple_returns(prices: &[f64]) -> Vec<f64> {
    if prices.len() < 2 { return Vec::new(); }
    let mut out = Vec::with_capacity(prices.len() - 1);
    for i in 1..prices.len() {
        let prev = prices[i - 1];
        out.push(if prev == 0.0 { 0.0 }
                 else { (prices[i] - prev) / prev });
    }
    out
}
```

Key Insight

The identity $\ln(1 + r^{\text{simple}}) = r^{\log}$ holds exactly. For small returns the two are nearly equal; for large returns they diverge. This identity is a cheap property test that catches implementation bugs in either function.

6.4. Moments

Central moment
$m_k = \frac{1}{n}\sum_{i=1}^{n}(x_i - \bar{x})^k$

Skewness
$g_1 = \frac{m_3}{m_2^{3/2}}$

Excess kurtosis
$g_2 = \frac{m_4}{m_2^2} - 3$

The k-th central moment is

$$m_k = \frac{1}{n}\sum_{i=1}^{n}(x_i - \bar{x})^k.$$

Skewness and excess kurtosis are standardised ratios of these moments:

$$g_1 = \frac{m_3}{m_2^{3/2}} \quad \text{(skewness)}$$
$$g_2 = \frac{m_4}{m_2^2} - 3 \quad \text{(excess kurtosis)}$$

The Gaussian distribution has $g_1 = 0$ and $g_2 = 0$. Positive skewness means a longer right tail; negative skewness a longer left tail. Positive excess kurtosis means heavier tails than Gaussian.

```rust
pub fn skewness(data: &[f64]) -> Result<f64, CoreError> {
    if data.len() < 3 { return Err(CoreError::InsufficientData { .. }); }
    let m = mean(data);
    let m2 = central_moment_2(data, m);
    let m3 = central_moment_3(data, m);
    Ok(m3 / m2.powf(1.5))
}

pub fn excess_kurtosis(data: &[f64]) -> Result<f64, CoreError> {
    if data.len() < 4 { return Err(CoreError::InsufficientData { .. }); }
    let m = mean(data);
    let m2 = central_moment_2(data, m);
    let m4 = central_moment_4(data, m);
    Ok(m4 / (m2 * m2) - 3.0)
}
```

Key Insight

We use the sample variance ($n-1$ denominator) for σ^2, consistent with `qf_common::compute_stats()`, but use population central moments

(denominator n) for m_2, m_3, m_4 in skewness and kurtosis. This is the textbook definition; the difference between the two vanishes for large n.

The `Moments` trait abstracts over any data source that can produce sample moments, so the same analytics work on slices, owned vectors, and `PriceSeries`:

```rust
pub trait Moments {
    fn mean(&self) -> f64;
    fn variance(&self) -> Result<f64, CoreError>;
    fn std_dev(&self) -> Result<f64, CoreError>;
    fn skewness(&self) -> Result<f64, CoreError>;
    fn excess_kurtosis(&self) -> Result<f64, CoreError>;
}

impl Moments for &[f64] { /* delegate to free functions */ }
impl Moments for Vec<f64> { /* delegate to slice impl */ }
impl Moments for PriceSeries { /* delegate to slice impl */ }
```

6.5. Rolling Windows

Rolling window
For window w on data of length n, the output has length $n - w + 1$ and entry i aggregates `data`$[i..i + w]$.

A rolling window applies an aggregation over each contiguous slice of length w. The generic `rolling` function takes a closure, so any aggregation (mean, std, max, custom) drops in:

```rust
pub fn rolling<F, T>(window: usize, data: &[f64], f: F)
    -> Result<Vec<T>, CoreError>
where F: Fn(&[f64]) -> T
{
    if window == 0 || window > data.len() {
        return Err(CoreError::InvalidWindow { window, len: data.len() });
    }
    let mut out = Vec::with_capacity(data.len() - window + 1);
    for i in 0..=(data.len() - window) {
        out.push(f(&data[i..i + window]));
    }
    Ok(out)
}

pub fn rolling_mean(window: usize, data: &[f64]) -> Result<Vec<f64>, CoreError> {
    rolling(window, data, mean)
}
```

6.6. Hand-Rolled Simulation

6.6.1. xorshift64

xorshift64
Three shifts and a multiply. Period $2^{64} - 1$. Fast, stateless, good enough for Monte Carlo when cryptographic strength is not required.

We implement Vigna's xorshift64 PRNG: three xor-shifts on a 64-bit state, then a multiply by a constant. A seed of zero produces a degenerate stream, so we replace it with a fixed default:

```rust
pub struct XorShift64 { state: u64 }

impl Rng for XorShift64 {
    fn next_u64(&mut self) -> u64 {
        let mut x = self.state;
        x ^= x >> 12;
        x ^= x << 25;
        x ^= x >> 27;
        self.state = x;
        x.wrapping_mul(0x2545_F491_4F6C_DD1D)
    }
    fn next_f64(&mut self) -> f64 {
        (self.next_u64() >> 11) as f64 / (1u64 << 53) as f64
    }
}
```

6.6.2. Box-Muller Normal

To sample $N(\mu, \sigma^2)$ without an external library, we use the Box-Muller transform: two independent uniforms $u_1, u_2 \sim U(0,1)$ produce one standard normal variate

$$Z = \sqrt{-2\ln u_1}\cos(2\pi u_2).$$

```rust
pub fn box_muller_normal<R: Rng + ?Sized>(
    rng: &mut R, mean: f64, std: f64,
) -> f64 {
    let u1 = rng.next_f64().max(f64::EPSILON);
    let u2 = rng.next_f64();
    let r = (-2.0 * u1.ln()).sqrt();
    let theta = 2.0 * PI * u2;
    mean + std * (r * theta.cos())
}
```

The `Distribution` trait lets us plug in new distributions (exponential, Student-t, jump-diffusion) without touching the simulator:

```rust
pub trait Distribution {
    fn sample<R: Rng + ?Sized>(&self, rng: &mut R) -> f64;
}

pub struct Normal { pub mean: f64, pub std: f64 }

impl Distribution for Normal {
    fn sample<R: Rng + ?Sized>(&self, rng: &mut R) -> f64 {
        box_muller_normal(rng, self.mean, self.std)
    }
}
```

6.6.3. Geometric Brownian Motion

Exact GBM solution

$S_{t+\Delta t} = S_t \exp\left(\left(\mu - \frac{1}{2}\sigma^2\right)\Delta t + \sigma\sqrt{\Delta t}\, Z\right)$

With $\sigma = 0$: $S_T = S_0 e^{\mu T}$, deterministic.

The stochastic differential equation

$$dS_t = \mu S_t\, dt + \sigma S_t\, dW_t$$

has the exact solution

$$S_{t+\Delta t} = S_t \exp\left(\left(\mu - \tfrac{1}{2}\sigma^2\right)\Delta t + \sigma\sqrt{\Delta t}\, Z\right),$$

where $Z \sim N(0,1)$. We simulate one step at a time:

```rust
pub fn gbm_paths<R: Rng + ?Sized>(
    s0: f64, mu: f64, sigma: f64, t: f64,
    n_steps: usize, n_paths: usize, rng: &mut R,
) -> Vec<Vec<f64>> {
    let dt = t / n_steps as f64;
    let drift = (mu - 0.5 * sigma * sigma) * dt;
    let diffusion = sigma * dt.sqrt();
    let normal = Normal::standard();
    let mut paths = Vec::with_capacity(n_paths);
    for _ in 0..n_paths {
        let mut path = Vec::with_capacity(n_steps + 1);
        path.push(s0);
        let mut s = s0;
        for _ in 0..n_steps {
            let z = normal.sample(rng);
            s *= (drift + diffusion * z).exp();
            path.push(s);
        }
        paths.push(path);
    }
    paths
}
```

Key Insight

With $\sigma = 0$, the Z term drops out and $S_T = S_0 \exp(\mu T)$ deterministically. This is a useful sanity check: a zero-volatility GBM should reproduce the risk-free growth exactly, with no RNG dependence.

6.7. Fat Tails

```
$ cargo run -p quant-core --example fat_tails
1. Pure Gaussian (10k N(0,1) draws)
  skewness     = +0.0267
  excess kurt  = +0.0323

2. Fat tails (every 100th draw x10)
  skewness     = -0.3051
  excess kurt  = +74.1272

3. GBM terminal prices (10k paths, 252 steps)
  skewness     = +0.5957
  excess kurt  = +0.5959
```

The pure Gaussian sample has excess kurtosis near 0, as expected. Scaling every 100th draw by 10 injects rare large moves; the excess kurtosis jumps from ~ 0 to ~ 74. GBM terminal prices are log-normally distributed, so they have positive excess kurtosis (heavier right tail than a Gaussian) and positive skewness — a direct consequence of the exponential in the exact solution.

Key Insight

A Gaussian model fit to fat-tailed data will price a 4-sigma event as a "1-in-15,000-years" occurrence. The true frequency under fat tails may be once a decade. This is why Value-at-Risk computed under Gaussian assumptions systematically underestimates tail losses, and why option-implied volatilities smile — the market knows the tails are fatter than Gaussian allows.

6.8. Exercises

1. **Sample vs population variance**: For $x \in \{1, 2, 3, 4, 5\}$, compute both the population variance (denominator n) and the sample variance (denominator $n - 1$). How large is the bias? When does it matter?
2. **Skewness sign**: Construct a synthetic series with negative skew (long left tail) and one with positive skew. Verify `skewness()` returns the correct sign. Which one is typical of equity returns?
3. **RNG period**: The xorshift64 generator has period $2^{64} - 1$. Write a test that verifies the first 1000 outputs from seed 42 are distinct, and reason about why "distinct" is necessary but not sufficient for a good PRNG.
4. **GBM convergence**: Simulate 10 000 GBM paths over one year (252 steps) with $S_0 = 100$, $\mu = 0.05$, $\sigma = 0.2$. Compute the mean terminal price. Compare with the analytical expectation $E[S_T] = S_0 e^{\mu T}$. How close is the Monte Carlo estimate?
5. **Jump-diffusion**: Extend `gbm_paths()` with a Poisson jump term: with probability $\lambda \Delta t$ per step, multiply S_t by e^J where $J \sim N(\mu_J, \sigma_J^2)$. How does the excess kurtosis of terminal prices change as λ grows?

Time Series: Stationarity and Fractional Differentiation

7.

7.1. Learning Objectives

Most financial time series are non-stationary: prices wander, volatilities climb and fall, autocorrelations decay slowly. Naive regression on non-stationary data produces spurious correlations and unstable forecasts. After completing this chapter you can:

- Fit an ordinary least squares regression by hand via Gaussian elimination with partial pivoting on the normal equations
- Compute standard errors, t-statistics, and R^2 without any external statistics crate
- Compute the sample autocorrelation function and read its structure
- Test for a unit root with the Augmented Dickey-Fuller test and interpret the MacKinnon critical value
- Apply López de Prado's fixed-width fractional differentiation to make a series stationary while preserving memory
- Find the minimum differencing order d that achieves stationarity on real and synthetic price series

Key Insight

Traditional differencing ($d = 1$) makes a price series stationary but erases all long-range memory — the resulting signal is essentially noise. Fractional differentiation ($0 < d < 1$) achieves stationarity while keeping as much memory as the data allow. The smallest such d is the sweet spot for ML features that predict future returns.

7.2. Stationarity

Weak stationarity

A process is weakly stationary if its mean is constant and its autocovariance depends only on the lag, not on time. "Stationary" in this chapter means weakly stationary.

A time series $\{y_t\}$ is *weakly stationary* if its first two moments are time-invariant: $E[y_t] = \mu$ and $\mathrm{Cov}(y_t, y_{t-k}) = \gamma_k$ depend only on the lag k. A random walk $y_t = y_{t-1} + \varepsilon_t$ fails this definition because its variance grows without bound. Non-stationary data break the assumptions behind OLS inference, forecast intervals, and most ML models.

The standard remedy is differencing: $\Delta y_t = y_t - y_{t-1}$. With $d = 1$ the series becomes stationary but loses all memory of the distant past. We need something in between.

7.3. OLS via Gaussian Elimination

Normal equations

$\hat{\beta} = (X'X)^{-1}X'y$. We never invert $X'X$ explicitly; we solve $X'X\hat{\beta} = X'y$ by Gaussian elimination.

Given a design matrix X (rows are observations, columns are features) and a response y, the OLS estimator minimises $\|y - X\beta\|^2$. The first-order condition gives the normal equations

$$(X'X)\,\hat{\beta} = X'y.$$

We solve this $k \times k$ linear system by Gaussian elimination with partial pivoting: at each column we swap the row with the largest absolute pivot into the diagonal position, then eliminate below.

```rust
pub fn ols(x: &[Vec<f64>], y: &[f64]) -> Result<OlsFit, TimeSeriesError> {
    let n = x.len();
    let k = x[0].len();
    // Normal equations: A = X'X, b = X'y.
    let mut a = vec![vec![0.0_f64; k]; k];
    let mut b = vec![0.0_f64; k];
    for (xi, yi) in x.iter().zip(y.iter()) {
        for i in 0..k {
            b[i] += xi[i] * yi;
            for j in 0..k {
                a[i][j] += xi[i] * xi[j];
            }
        }
    }
    let coeffs = gauss_solve(&mut a, &mut b)?;
    // residuals, SSE, SST, R^2, standard errors, t-stats ...
}
```

Key Insight

Partial pivoting matters. Without it, a small diagonal entry relative to the entries below causes catastrophic cancellation when we eliminate. Pivoting costs nothing (a row swap per column) and lifts the worst-case error from exponential in k to roughly k machine epsilons.

Standard errors require the diagonal of $(X'X)^{-1}$. Rather than inverting the full matrix, we solve $X'X\, e_j = \text{unit}_j$ for each column j and read off $e_j[j] = (X'X)^{-1}_{jj}$. The standard error is

$$\text{SE}(\hat{\beta}_j) = \sqrt{(X'X)^{-1}_{jj}\, s^2}, \quad s^2 = \frac{\text{SSE}}{n-k}.$$

The t-statistic is $\hat{\beta}_j / \text{SE}(\hat{\beta}_j)$ and $R^2 = 1 - \text{SSE}/\text{SST}$.

7.4. Autocorrelation Function

Sample ACF

$\hat{\rho}_k = \dfrac{\sum_{t=k+1}^{n}(y_t - \bar{y})(y_{t-k} - \bar{y})}{\sum_{t=1}^{n}(y_t - \bar{y})^2}$, with $\hat{\rho}_0 = 1$ by construction.

The autocorrelation at lag k is the correlation between y_t and y_{t-k}. White noise has $\hat{\rho}_k \approx 0$ for $k > 0$; an AR(1) with coefficient ϕ has $\hat{\rho}_k \approx \phi^k$; a random walk has $\hat{\rho}_k \approx 1$ for all small k.

```rust
pub fn acf(data: &[f64], max_lag: usize)
    -> Result<Vec<f64>, TimeSeriesError>
{
    let n = data.len();
    let ybar: f64 = data.iter().sum::<f64>() / n as f64;
    let denom: f64 = data.iter()
        .map(|&v| { let d = v - ybar; d * d })
        .sum();
    let mut out = Vec::with_capacity(max_lag + 1);
    for k in 0..=max_lag {
        if k == 0 { out.push(1.0); continue; }
        let cov: f64 = (k..n)
            .map(|t| (data[t] - ybar) * (data[t - k] - ybar))
            .sum();
        out.push(cov / denom);
    }
    Ok(out)
}
```

7.5. Augmented Dickey-Fuller Test

ADF regression

$\Delta y_t = \alpha + \gamma\, y_{t-1} + \sum_{i=1}^{p} \delta_i\, \Delta y_{t-i} + \varepsilon_t$.

Reject $H_0 : \gamma = 0$ (unit root) when the t-ratio on $\hat{\gamma}$ is below the MacKinnon 5% critical value −2.86.

The ADF test regresses the first difference on the lagged level and lagged differences. The null hypothesis is a unit root ($\gamma = 0$); the alternative is stationarity ($\gamma < 0$). Because the test statistic does not follow a standard distribution under the null, we compare it to MacKinnon's simulated critical values. For a constant-only specification at 5% significance the value is −2.86.

```rust
pub const MACKINNON_5PCT: f64 = -2.86;

pub fn adf_test(data: &[f64], lags: usize)
    -> Result<AdfResult, TimeSeriesError>
{
    let n = data.len();
    let dy: Vec<f64> = (1..n).map(|t| data[t] - data[t - 1]).collect();
    // Rows: [1, y_{t-1}, dy[t-1], dy[t-2], ..., dy[t-lags]]
    let mut x: Vec<Vec<f64>> = Vec::new();
    let mut y: Vec<f64> = Vec::new();
    for t in (lags + 1)..n {
        let mut row = Vec::with_capacity(lags + 2);
        row.push(1.0);
        row.push(data[t - 1]);
        for i in 1..=lags { row.push(dy[t - 1 - i]); }
        y.push(dy[t - 1]);
        x.push(row);
    }
    let fit = ols(&x, &y)?;
    let stat = fit.t_stats[1]; // gamma is index 1 (after intercept)
    Ok(AdfResult {
        statistic: stat,
        critical_value: MACKINNON_5PCT,
        lags,
        is_stationary: stat < MACKINNON_5PCT,
    })
}
```

Key Insight

The lagged-level coefficient γ is the key. If $\gamma = 0$, shocks to y_t are permanent (unit root). If $\gamma < 0$, shocks decay at rate $1 + \gamma$ per period. The more negative γ, the faster the series reverts to the mean — and the more strongly stationary.

7.6. Fractional Differentiation

Binomial series

$(1 - B)^d = \sum_{k=0}^{\infty} w_k B^k$, where $w_0 = 1$ and $w_k = -w_{k-1}\,\dfrac{d - k + 1}{k}$.

For $d = 1$: $w = [1, -1, 0, \ldots]$.
For $d \in (0, 1)$: infinitely many non-zero weights decaying in magnitude.

The backward shift B satisfies $By_t = y_{t-1}$. The first difference is $(1 - B)y_t$. Fractional differentiation generalises this to $(1 - B)^d$ for any real $d \in [0, 2]$. The binomial series gives the weights:

$$w_0 = 1,$$
$$w_k = -w_{k-1}\,\frac{d - k + 1}{k} \quad (k \geq 1).$$

For $d = 1$ the weights are $[1, -1, 0, 0, \ldots]$ — the ordinary first difference. For $0 < d < 1$ infinitely many weights are non-zero, with $|w_k|$ decaying roughly as $k^{-(1+d)}$. We truncate when $|w_k| <$ threshold (the fixed-width window of López de Prado, AFML Ch.5):

```rust
pub fn ffd_weights(d: f64, threshold: f64)
    -> Result<Vec<f64>, TimeSeriesError>
{
    let mut weights = vec![1.0_f64];
    let mut w = 1.0_f64;
```

```
    let mut k = 1_usize;
    loop {
        w = -w * (d - (k as f64) + 1.0) / k as f64;
        if w.abs() < threshold { break; }
        weights.push(w);
        if k > 10_000 { break; }
        k += 1;
    }
    Ok(weights)
}
```

Key Insight

The loop breaks *before* pushing a weight below the threshold. This makes $d = 0$ yield $[1.0]$ (identity, full memory) and $d = 1$ yield $[1.0, -1.0]$ exactly (ordinary first difference, no memory). Every intermediate d keeps a finite prefix of the binomial series.

Applying the window is a weighted moving average over the last `weights.len()` observations:

```
pub fn frac_diff(data: &[f64], d: f64, threshold: f64)
    -> Result<Vec<f64>, TimeSeriesError>
{
    let weights = ffd_weights(d, threshold)?;
    let wlen = weights.len();
    let mut out = Vec::with_capacity(data.len() - wlen + 1);
    for t in (wlen - 1)..data.len() {
        let acc: f64 = weights.iter()
            .enumerate()
            .map(|(k, &w)| w * data[t - k])
            .sum();
        out.push(acc);
    }
    Ok(out)
}
```

7.6.1. Finding the Minimum d

Binary search over $d \in [0, 1]$ returns the smallest order for which the ADF test rejects the unit root. We test the endpoints first: if $d = 0$ already passes, no differencing is needed; if $d = 1$ fails, we return 1.0 conservatively.

```
pub fn find_min_d(data: &[f64], threshold: f64, tolerance: f64)
    -> Result<f64, TimeSeriesError>
{
    let passes = |d: f64| {
        let diff = frac_diff(data, d, threshold)?;
        Ok(adf_test(&diff, 1)?.is_stationary)
    };
    if passes(0.0)? { return Ok(0.0); }
    if !passes(1.0)? { return Ok(1.0); }
    let (mut lo, mut hi) = (0.0_f64, 1.0_f64);
    while hi - lo > tolerance {
        let mid = 0.5 * (lo + hi);
        if passes(mid)? { hi = mid; } else { lo = mid; }
    }
    Ok(hi)
}
```

7.7. The Stationarity-Memory Tradeoff in Action

```
$ cargo run -p quant-timeseries --example stationarity
==============================
Stationarity Analysis
==============================

Random Walk (500 steps):
  ADF statistic: -1.1064
  Critical (5%):  -2.86
```

```
  Stationary:     NO

First Difference (d=1):
  ADF statistic: -15.8418
  Stationary:    YES
  ACF(1):        -0.0269 (memory destroyed)

Fractional Diff (d=0.4):
  ADF statistic: -4.8991
  Stationary:    YES
  ACF(1):         0.6878 (memory preserved)
==============================
```

The random walk has an ADF statistic of -1.1, well above the -2.86 critical value: we cannot reject the unit root. First differencing fixes stationarity (statistic plunges to -15.8) but the ACF at lag 1 is essentially zero — all memory is gone. Fractional differentiation with $d = 0.4$ also rejects the unit root, but the ACF at lag 1 is 0.69: most of the long-range information survives.

Key Insight

This is the core finding of AFML Ch.5: there exists a $d^\star \in (0, 1)$ that simultaneously satisfies the stationarity requirement of statistical inference *and* the memory requirement of predictive ML features. Any $d < d^\star$ is non-stationary; any $d > d^\star$ throws away more memory than necessary.

7.8. Exercises

1. **Partial autocorrelation**: Implement the PACF via the Yule-Walker equations. For an AR(p) process, PACF should cut off after lag p. Verify on simulated AR(2) data with $\phi_1 = 0.5$, $\phi_2 = -0.3$.
2. **KPSS test**: The KPSS test has the opposite null hypothesis to ADF — H_0 is stationarity. Implement it and compare the conclusions on white noise, random walk, and fractionally differenced series. Where do the two tests disagree?
3. **Real returns**: Generate a long GBM price path and find the minimum d for stationarity. Repeat for different volatilities $\sigma \in \{0.1, 0.2, 0.4\}$. Does σ affect $d^\star$?
4. **Expanding-window frac_diff**: The fixed-width window drops the first $K - 1$ observations. Implement an expanding-window variant that uses all available history for the early points (weights are truncated, not zero). What is the tradeoff versus fixed-width?
5. **Lag selection for ADF**: The AIC and BIC criteria pick the number of lagged differences automatically. Implement AIC-based lag selection and check whether the stationarity conclusion changes for your test series.

8. Volatility Models: EWMA, ARCH, GARCH

Chapter content pending — Phase 8 implementation.

9. Stochastic Processes and Monte Carlo

Chapter content pending — Phase 9 implementation.

Derivatives and Portfolio Theory

10. Options Pricing: Black-Scholes and Beyond

Chapter content pending — Phase 10 implementation.

11. Portfolio Optimization: Markowitz and Beyond

Chapter content pending — Phase 11 implementation.

12. Factor Models and PCA

Chapter content pending — Phase 12 implementation.

Advanced Topics

13. Market Microstructure

Chapter content pending — Phase 13 implementation.

14. AFML: From Research to Production

Chapter content pending — Phase 14 implementation.

Appendices

A. Mathematical Notation

Appendix content pending.

Rust Patterns for Finance

B.

Appendix content pending.

C. Dataset Sources

Appendix content pending.